

Chapter 1: Review of the Fundamentals - Python Programming

Tutorial by Ayes Chinmay

July 2025

Contents

✓1 Introduction	2
✓2 Python Quick Start	2
✓3 Variables and Naming Names	2
✓4 Combined Assignment Operators	2
✓5 Summary of Python Arithmetic Operators	3
✓6 Elementary Data Types: Integer and Floating Point	3
✓7 Basic Input and Output	3
✓8 Function Definitions	3
✓9 The Python “if” Statement	4
10 The Python “while” Statement	4
11 A Couple of Cool Little Apps	4
12 Summary of Python Boolean Operators	4
13 Function Arguments and Return Values	5
14 The Forward Reference Problem	5
15 Python Strings	5
16 Python Lists (and a Cool Sorting App)	5
17 The “for” Statement and Ranges	6
18 Tuples	6
19 Dictionaries	6
20 Sets	6
21 Global and Local Variables	6

1 Introduction

This tutorial covers the fundamentals of Python programming as outlined in Chapter 1. It is designed for beginners and provides detailed explanations, examples, and exercises for each topic.

2 Python Quick Start

Python is a versatile, readable programming language. To start:

1. Download Python from <https://www.python.org/downloads/> (e.g., Python 3.12).
2. Install and verify by running `python -version` in a terminal.
3. Use an editor like IDLE or Visual Studio Code.

Example: Hello, World!

```
1 print("Hello, World!")
```

Run this in IDLE or save as `hello.py` and execute with `python hello.py`.

3 Variables and Naming Names

Variables store data. Use meaningful names following these rules:

- Start with a letter or underscore (e.g., `age`, `_count`).
- Use letters, numbers, or underscores (e.g., `user2`).
- Case-sensitive (e.g., `Age` $\neq$ `age`).

Example:

```
1 name = "Alice"  
2 age = 25
```

4 Combined Assignment Operators

These combine arithmetic and assignment:

- `+=`: Add and assign (e.g., `x += 5` is `x = x + 5`).
- `-=`, `*=`, `/=`, `//`, `%=`, `**=`.

Example:

```
1 x = 10  
2 x += 5 # x is now 15  
3 x *= 2 # x is now 30  
4 print(x)
```

5 Summary of Python Arithmetic Operators

Python supports:

- `+`: Addition
- `-`: Subtraction
- `*`: Multiplication
- `/`: Division
- `//`: Integer division
- `%`: Modulus
- `**`: Exponentiation

Example:

```
1 a = 10
2 b = 3
3 print(a + b, a / b, a // b, a ** 2) # 13, 3.333..., 3, 100
```

6 Elementary Data Types: Integer and Floating Point

- **Integer**: Whole numbers (e.g., 42, -7).
- **Float**: Decimal numbers (e.g., 3.14, 0.001).

Example:

```
1 integer = 100
2 floating = 2.5
3 print(integer * floating) # 250.0
```

7 Basic Input and Output

Use `print()` for output and `input()` for user input. Example:

```
1 name = input("Enter your name: ")
2 print("Hello, " + name)
```

Output (if user enters "Alice"):

```
Enter your name: Alice
Hello, Alice
```

8 Function Definitions

Functions are reusable code blocks defined with `def`. Example:

```
1 def greet(name):
2     return "Hello, " + name
3
4 print(greet("Bob")) # Hello, Bob
```

9 The Python "if" Statement

The if statement controls flow based on conditions. Example:

```
1 age = 18
2 if age >= 18:
3     print("Adult")
4 else:
5     print("Minor")
```

10 The Python "while" Statement

A while loop repeats while a condition is true. Example:

```
1 count = 1
2 while count <= 5:
3     print(count)
4     count += 1
```

Output: 1 2 3 4 5

11 A Couple of Cool Little Apps

App 1: Simple Calculator

```
1 num1 = float(input("Enter first number: "))
2 num2 = float(input("Enter second number: "))
3 op = input("Enter operator (+, -, *, /): ")
4 if op == "+":
5     print(num1 + num2)
6 elif op == "-":
7     print(num1 - num2)
8 elif op == "*":
9     print(num1 * num2)
10 elif op == "/":
11     print(num1 / num2)
```

App 2: Number Guessing Game

```
1 import random
2 number = random.randint(1, 10)
3 guess = int(input("Guess a number (1-10): "))
4 while guess != number:
5     guess = int(input("Wrong! Try again: "))
6 print("Correct!")
```

12 Summary of Python Boolean Operators

- and, or, not
- Comparison: ==, !=, <, >, <=, >=

Example:

```

1 x = 5
2 y = 10
3 print(x < y and y > 0) # True

```

13 Function Arguments and Return Values

Functions can take multiple arguments and return values. Example:

```

1 def add(a, b):
2     return a + b
3
4 result = add(3, 4)
5 print(result) # 7

```

14 The Forward Reference Problem

Python requires functions to be defined before they are called. Example (will cause an error):

```

1 print(my_func()) # Error: my_func not defined yet
2 def my_func():
3     return "Hello"

```

Fix: Define the function first.

15 Python Strings

Strings are sequences of characters, enclosed in "" or ". Example:

```

1 text = "Python"
2 print(text[0]) # P
3 print(text.upper()) # PYTHON
4 print(len(text)) # 6

```

16 Python Lists (and a Cool Sorting App)

Lists are ordered, mutable collections. Example:

```

1 numbers = [5, 2, 8, 1]
2 numbers.sort()
3 print(numbers) # [1, 2, 5, 8]

```

Sorting App

```

1 numbers = []
2 while True:
3     num = input("Enter a number (or 'done'): ")
4     if num == "done":
5         break
6     numbers.append(float(num))
7 numbers.sort()
8 print("Sorted:", numbers)

```

17 The "for" Statement and Ranges

Use for loops to iterate over sequences. Example:

```
1 for i in range(1, 6):  
2     print(i) # 1 2 3 4 5
```

18 Tuples

can not change the value

Tuples are immutable sequences. Example:

```
1 point = (3, 4)  
2 print(point[0]) # 3
```

19 Dictionaries

Dictionaries store key-value pairs. Example:

```
1 student = {"name": "Alice", "age": 20}  
2 print(student["name"]) # Alice  
3 student["grade"] = "A"  
4 print(student)
```

20 Sets

Sets store unique, unordered elements. Example:

```
1 numbers = {1, 2, 2, 3}  
2 print(numbers) # {1, 2, 3}  
3 numbers.add(4)  
4 print(numbers)
```

21 Global and Local Variables

Local variables are defined inside functions; global variables are outside. Example:

```
1 x = 10 # Global  
2 def my_func():  
3     x = 5 # Local  
4     print(x) # 5  
5 my_func()  
6 print(x) # 10
```

22 Exercises

1. Write a program that takes two numbers as input and prints their sum, difference, product, and quotient.
2. Create a function that checks if a number is even or odd.

3. Write a program that stores 5 names in a list, sorts them, and prints the sorted list.
4. Create a dictionary to store 3 students' names and grades, then print each student's information.

23 Conclusion

This tutorial covered the core concepts of Python programming, from variables and data types to control structures, functions, and data structures. Practice the exercises and refer to <https://docs.python.org/3/> for further exploration.

✓ — are important

Chapter 2: Advanced String Capabilities in Python

Tutorial by Ayes Chinmay

July 22, 2025

Contents

1	Introduction	2
2	Strings Are Immutable	2
3	Numeric Conversions, Including Binary	2
4	String Operators (+, =, *, >, etc.)	2
5	Indexing and Slicing	3
6	Single-Character Functions (Character Codes)	3
7	Building Strings Using “join”	3
8	Important String Functions	3
9	Binary, Hex, and Octal Conversion Functions	4
10	Simple Boolean (“is”) Methods	4
11	Case Conversion Methods	4
12	Search-and-Replace Methods	5
13	Breaking Up Input Using “split”	5
14	Stripping	5
15	Justification Methods	6
16	Summary	6
17	Answers to Exercises	6

1 Introduction

This tutorial covers Chapter 2, "Advanced String Capabilities," focusing on Python's powerful string manipulation features. You'll learn about string immutability, conversions, operators, indexing, and various string methods. Each section includes explanations, code examples, and exercises.

2 Strings Are Immutable

Strings in Python are immutable, meaning their contents cannot be changed after creation. Any modification creates a new string.

```
1 text = "Hello"
2 text = text + " World" # Creates a new string
3 print(text)
```

Output:

Hello World

Exercise 1: Create a string greeting = "Hi" and append " there!" to it. Print the result.

3 Numeric Conversions, Including Binary

Convert strings to numbers using `int()`, `float()`, and convert numbers to binary strings using `bin()`.

```
1 num_str = "123"
2 num = int(num_str)
3 binary = bin(num)[2:] # Remove '0b' prefix
4 print(f"Number: {num}, Binary: {binary}")
```

Output:

Number: 123, Binary: 1111011

Exercise 2: Convert the string "255" to an integer and print its binary representation.

4 String Operators (+, =, *, >, etc.)

String operators include concatenation (+), repetition (*), and comparison (>, <, ==).

```
1 str1 = "Hello"
2 str2 = "World"
3 concat = str1 + " " + str2
4 repeat = str1 * 2
5 print(f"Concatenation: {concat}")
6 print(f"Repetition: {repeat}")
7 print(f"Is str1 > str2? {str1 > str2}")
```

Output:

Concatenation: Hello World

Repetition: HelloHello

Is str1 > str2? False

Exercise 3: Concatenate two strings and repeat the result three times.

5 Indexing and Slicing

Strings are indexed (0-based) and can be sliced using [start:end:step].

```
1 text = "Python"
2 print(f"First char: {text[0]}")
3 print(f"Slice [1:4]: {text[1:4]}")
4 print(f"Reverse: {text[::-1]}")
```

Output:

First char: P
Slice [1:4]: yth
Reverse: nohtyP

Exercise 4: Extract the substring "gram" from "Programming".

6 Single-Character Functions (Character Codes)

Use ord() to get a character's Unicode code point and chr() to convert a code point to a character.

```
1 char = "A"
2 code = ord(char)
3 back = chr(code)
4 print(f"Character {char} has code {code}")
5 print(f"Code {code} is character {back}")
```

Output:

Character A has code 65
Code 65 is character A

Exercise 5: Find the Unicode code point of 'z' and convert 97 back to a character.

7 Building Strings Using "join"

The join() method concatenates a list of strings with a separator.

```
1 words = ["Hello", "World"]
2 sentence = " ".join(words)
3 print(sentence)
```

Output:

Hello World

Exercise 6: Join the list ["apple", "banana", "orange"] with a comma and space.

8 Important String Functions

Key string methods include len(), upper(), lower(), and strip().

```
1 text = " Python "
2 print(f"Length: {len(text)}")
3 print(f"Upper: {text.upper()}")
```

Advanced String Capabilities in Python - Chapter 2 Tutorial

```
4 print(f"Lower: {text.lower()}")
5 print(f"Stripped: '{text.strip()}'")
```

Output:

Length: 9
Upper: PYTHON
Lower: python
Stripped: 'Python'

Exercise 7: Apply `upper()` and `strip()` to "hello" and print the result.

9 Binary, Hex, and Octal Conversion Functions

Convert numbers to binary (`bin()`), hexadecimal (`hex()`), or octal (`oct()`).

```
1 num = 42
2 print(f"Binary: {bin(num)[2:]}")
3 print(f"Hex: {hex(num)[2:]}")
4 print(f"Octal: {oct(num)[2:]}")
```

Output:

Binary: 101010
Hex: 2a
Octal: 52

Exercise 8: Convert 100 to hexadecimal and octal.

10 Simple Boolean ("is") Methods

Methods like `isalpha()`, `isdigit()`, and `isspace()` test string properties.

```
1 text = "Python123"
2 print(f"Is alpha? {text.isalpha()}")
3 print(f"Is digit? {text.isdigit()}")
4 print(f"Is alphanumeric? {text.isalnum()}")
```

Output:

Is alpha? False
Is digit? False
Is alphanumeric? True

Exercise 9: Check if "123" is a digit and if "abc" is alphabetic.

11 Case Conversion Methods

Methods like `upper()`, `lower()`, `title()`, and `capitalize()` modify string case.

```
1 text = "hello world"
2 print(f"Title: {text.title()}")
3 print(f"Capitalize: {text.capitalize()}")
```

Output:

Title: Hello World
Capitalize: Hello world

Exercise 10: Convert "python programming" to title case.

12 Search-and-Replace Methods

Use `find()`, `replace()`, and `count()` for searching and replacing.

```
1 text = "Hello World"
2 print(f"Find 'World': {text.find('World')}")
3 print(f"Replace: {text.replace('World', 'Python')}")
4 print(f"Count 'l': {text.count('l')}")
```

Output:

Find 'World': 6
Replace: Hello Python
Count 'l': 3

Exercise 11: Replace "cat" with "dog" in "The cat is on the mat".

13 Breaking Up Input Using "split"

The `split()` method splits a string into a list based on a delimiter.

```
1 text = "apple,banana,orange"
2 fruits = text.split(",")
3 print(fruits)
```

Output:

['apple', 'banana', 'orange']

Exercise 12: Split "one two three" on spaces.

14 Stripping

Methods like `strip()`, `lstrip()`, and `rstrip()` remove whitespace.

```
1 text = " Hello "
2 print(f"Strip: '{text.strip()}'")
3 print(f"Left strip: '{text.lstrip()}'")
4 print(f"Right strip: '{text.rstrip()}'")
```

Output:

Strip: 'Hello'
Left strip: 'Hello '
Right strip: ' Hello'

Exercise 13: Strip whitespace from " Python ".

15 Justification Methods

Methods like `center()`, `ljust()`, and `rjust()` align text.

```
1 text = "Python"
2 print(f"Center: '{text.center(10, '**)}'")
3 print(f"Left justify: '{text.ljust(10)}'")
4 print(f"Right justify: '{text.rjust(10)}'")
```

Output:

```
Center: '**Python**'
Left justify: 'Python  '
Right justify: '   Python'
```

Exercise 14: Center "Code" in a 10-character field with dashes.

16 Summary

This chapter covered advanced string manipulation in Python, including immutability, conversions, operators, indexing, and various methods for processing strings. Practice the exercises to master these concepts.

17 Answers to Exercises

1. Exercise 1:

```
1 greeting = "Hi"
2 greeting = greeting + " there!"
3 print(greeting)
```

2. Exercise 2:

```
1 num_str = "255"
2 num = int(num_str)
3 print(bin(num)[2:])
```

3. Exercise 3:

```
1 str1 = "Hello"
2 str2 = "World"
3 result = (str1 + " " + str2) * 3
4 print(result)
```

4. Exercise 4:

```
1 text = "Programming"
2 print(text[3:7])
```

5. Exercise 5:

```
1 print(ord('z'))
2 print(chr(97))
```

6. Exercise 6:

```
1 fruits = ["apple", "banana", "orange"]  
2 print(", ".join(fruits))
```

7. Exercise 7:

```
1 text = "  hello  "  
2 print(text.strip().upper())
```

8. Exercise 8:

```
1 num = 100  
2 print(hex(num)[2:])  
3 print(oct(num)[2:])
```

9. Exercise 9:

```
1 print("123".isdigit())  
2 print("abc".isalpha())
```

10. Exercise 10:

```
1 text = "python programming"  
2 print(text.title())
```

11. Exercise 11:

```
1 text = "The cat is on the mat"  
2 print(text.replace("cat", "dog"))
```

12. Exercise 12:

```
1 text = "one two three"  
2 print(text.split())
```

13. Exercise 13:

```
1 text = "  Python  "  
2 print(text.strip())
```

14. Exercise 14:

```
1 text = "Code"  
2 print(text.center(10, '-'))
```

Chapter 3: Advanced List Capabilities in Python

A Comprehensive Tutorial

Created by Ayes Chinmay

July 23, 2025

1 Introduction

This tutorial covers Chapter 3, 'Advanced List Capabilities,' from a Python programming textbook. It explores advanced features of Python lists, including creation, indexing, slicing, operators, functions, methods, and advanced concepts like list comprehension, stacks, and multidimensional lists. Each section includes explanations and practical examples to help beginners master these concepts.

2 Creating and Using Python Lists

Lists in Python are versatile, ordered, mutable collections. They can store elements of different types.

```
1 # Creating a list
2 fruits = ["apple", "banana", 42, 3.14]
3
4 # Accessing elements
5 print(fruits[0]) # Output: apple
6
7 # Modifying elements
8 fruits[1] = "orange"
9 print(fruits) # Output: ['apple', 'orange', 42, 3.14]
```

Lists are defined using square brackets, and elements are accessed or modified using indices.

3 Copying Lists Versus Copying List Variables

Assigning a list to a new variable creates a reference, not a copy. Use the `copy()` method or slicing for a new list.

```
1 original = [1, 2, 3]
2 reference = original # Points to the same list
3 copy = original.copy() # Creates a new list
4
5 reference[0] = 99
6 print(original) # Output: [99, 2, 3]
7 print(copy) # Output: [1, 2, 3]
```

Modifying reference changes original, but copy remains unchanged.

4 Indexing

4.1 Positive Indexes

Positive indices start at 0 for the first element.

```
1 numbers = [10, 20, 30, 40]
2 print(numbers[2]) # Output: 30
```

4.2 Negative Indexes

Negative indices count from the end, starting at -1.

```
1 print(numbers[-1]) # Output: 40
2 print(numbers[-2]) # Output: 30
```

4.3 Generating Index Numbers Using enumerate

The `enumerate()` function provides index-element pairs.

```
1 for index, fruit in enumerate(fruits):
2     print(f"Index {index}: {fruit}")
3 # Output:
4 # Index 0: apple
5 # Index 1: orange
6 # Index 2: 42
7 # Index 3: 3.14
```

5 Getting Data from Slices

Slicing extracts a portion of a list using `[start:end:step]`.

```
1 numbers = [0, 1, 2, 3, 4, 5]
2 print(numbers[1:4]) # Output: [1, 2, 3]
3 print(numbers[:2]) # Output: [0, 2, 4]
```

The end index is exclusive, and step specifies the increment.

6 Assigning into Slices

You can assign values to a slice to modify multiple elements.

```
1 numbers[1:3] = [10, 20]
2 print(numbers) # Output: [0, 10, 20, 3, 4, 5]
```

The assigned list must match the slice length or be iterable.

7 List Operators

Common operators include:

- `+`: Concatenates lists
- `*`: Repeats a list
- `in`: Checks for membership

```
1 list1 = [1, 2]
2 list2 = [3, 4]
3 print(list1 + list2) # Output: [1, 2, 3, 4]
```

```
1 print(list1 * 2) # Output: [1, 2, 1, 2]
2 print(2 in list1) # Output: True
```

8 Shallow Versus Deep Copying

Shallow copying (e.g., `copy()`) copies the list but not nested objects. Deep copying (`deepcopy()`) copies everything.

```
1 from copy import deepcopy
2 nested = [[1, 2], [3, 4]]
3 shallow = nested.copy()
4 deep = deepcopy(nested)
5
6 shallow[0][0] = 99
7 print(nested) # Output: [[99, 2], [3, 4]]
8 print(deep) # Output: [[1, 2], [3, 4]]
```

9 List Functions

Common list functions include `len()`, `min()`, `max()`, and `sum()`.

```
1 numbers = [1, 2, 3]
2 print(len(numbers)) # Output: 3
3 print(min(numbers)) # Output: 1
4 print(sum(numbers)) # Output: 6
```

10 List Methods: Modifying a List

Methods like `append()`, `insert()`, and `remove()` modify lists.

```
1 fruits = ["apple", "banana"]
2 fruits.append("cherry")
3 fruits.insert(1, "orange")
4 fruits.remove("banana")
5 print(fruits) # Output: ["apple", "orange", "cherry"]
```

11 List Methods: Getting Information on Contents

Methods like `count()` and `index()` provide information.

```
1 numbers = [1, 2, 1, 3]
2 print(numbers.count(1)) # Output: 2
3 print(numbers.index(2)) # Output: 1
```

12 List Methods: Reorganizing

Methods like `sort()` and `reverse()` reorganize lists.

```
1 numbers = [3, 1, 2]
2 numbers.sort()
3 print(numbers) # Output: [1, 2, 3]
4 numbers.reverse()
5 print(numbers) # Output: [3, 2, 1]
```

13 Lists as Stacks: RPN Application

Lists can act as stacks (LIFO) using `append()` and `pop()`. Reverse Polish Notation (RPN) uses a stack for calculations.

```
1 def rpn(expression):
2     stack = []
3     for token in expression.split():
4         if token in "+-*/":
5             b = stack.pop()
6             a = stack.pop()
7             if token == "+": stack.append(a + b)
8             elif token == "-": stack.append(a - b)
9             elif token == "*": stack.append(a * b)
10            elif token == "/": stack.append(a / b)
11        else:
12            stack.append(float(token))
13    return stack[0]
14
15 print(rpn("3 4 +")) # Output: 7.0
```

14 The reduce Function

The `reduce()` function from `functools` applies a function cumulatively to a list.

```
1 from functools import reduce
2 numbers = [1, 2, 3, 4]
3 sum_result = reduce(lambda x, y: x + y, numbers)
4 print(sum_result) # Output: 10
```

15 Lambda Functions

Lambda functions are anonymous functions defined with `lambda`.

```
1 double = lambda x: x * 2
2 print(double(5)) # Output: 10
```

16 List Comprehension

List comprehensions create lists concisely.

```
1 squares = [x**2 for x in range(5)]  
2 print(squares) # Output: [0, 1, 4, 9, 16]
```

17 Dictionary and Set Comprehension

Similar to list comprehensions, these create dictionaries or sets.

```
1 square_dict = {x: x**2 for x in range(3)}  
2 print(square_dict) # Output: {0: 0, 1: 1, 2: 4}  
3  
4 evens = {x for x in range(10) if x % 2 == 0}  
5 print(evens) # Output: {0, 2, 4, 6, 8}
```

18 Passing Arguments Through a List

Use `*` to unpack a list as function arguments.

```
1 def add(a, b, c):  
2     return a + b + c  
3  
4 numbers = [1, 2, 3]  
5 print(add(*numbers)) # Output: 6
```

19 Multidimensional Lists

19.1 Unbalanced Matrices

Multidimensional lists can have sublists of different lengths.

```
1 matrix = [[1, 2], [3, 4, 5], [6]]  
2 print(matrix[1][2]) # Output: 5
```

19.2 Creating Arbitrarily Large Matrices

Use list comprehension for uniform matrices.

```
1 rows, cols = 3, 4  
2 matrix = [[0 for _ in range(cols)] for _ in range(rows)]  
3 print(matrix) # Output: [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0,  
4     0]]
```

20 Summary

This chapter covered advanced list operations in Python, including creation, indexing, slicing, operators, copying, functions, methods, and advanced techniques like list comprehension, stacks, and multidimensional lists. These tools enable efficient data manipulation and complex program structures.

21 Review Questions

1. What is the difference between shallow and deep copying?
2. How does `enumerate()` assist in iterating over a list?
3. What is the purpose of list comprehension?
4. How can a list be used as a stack?
5. Explain the `reduce()` function with an example.

22 Suggested Problems

1. Write a program to reverse a list using slicing.
2. Create a list comprehension to generate a list of even numbers from 1 to 20.
3. Implement a program to transpose a 3x3 matrix using list comprehension.

23 Sample Solution: Matrix Transpose

Heres a solution to the third suggested problem:

```
1 matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2 transposed = [[matrix[j][i] for j in range(3)] for i in range(3)]
3 print(transposed) # Output: [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
```

This solution uses nested list comprehension to swap rows and columns of a 3x3 matrix.

Chapter 4: Shortcuts, Command Line, and Packages in Python

Tutorial by Ayes Chinmay

July 24, 2025

1 Overview

This tutorial covers Chapter 4, “Shortcuts, Command Line, and Packages,” from an intermediate Python programming text. It explores Python programming shortcuts, command-line usage, package management, and advanced concepts like decorators and generators. Each section includes explanations, code examples, and exercises to reinforce learning.

2 Twenty-Two Programming Shortcuts

Python offers concise ways to write efficient code. This section covers 22 shortcuts to enhance productivity.

2.1 Use Python Line Continuation as Needed

Use a backslash (\) to continue long lines, or rely on Python’s implicit continuation in parentheses.

```
1 # Explicit continuation
2 total = 1 + 2 + 3 + \
3     4 + 5
4
5 # Implicit continuation
6 numbers = (1 + 2 + 3 +
7     4 + 5)
8 print(total, numbers) # Output: 15 15
```

2.2 Use “for” Loops Intelligently

Leverage Python’s for loops with range() or iterables to avoid manual indexing.

```
1 # Instead of manual indexing
2 lst = ['a', 'b', 'c']
3 for item in lst:
4     print(item) # Output: a b c
```

Exercise: Rewrite a loop that prints indices and values using enumerate().

2.3 Understand Combined Operator Assignment (+= etc.)

Use operators like +=, -=, *= for concise updates.

```
1 total = 0
2 total += 5 # Equivalent to total = total + 5
3 print(total) # Output: 5
```

2.4 Use Multiple Assignment

Assign multiple variables in one line.

```
1 x, y, z = 1, 2, 3
2 print(x, y, z) # Output: 1 2 3
```

2.5 Use Tuple Assignment

Swap values or unpack tuples easily.

```
1 a, b = 10, 20
2 a, b = b, a # Swap values
3 print(a, b) # Output: 20 10
```

2.6 Use Advanced Tuple Assignment

Unpack nested structures or use * for variable-length unpacking.

```
1 a, *b, c = (1, 2, 3, 4, 5)
2 print(a, b, c) # Output: 1 [2, 3, 4] 5
```

Exercise: Unpack a list into first, middle, and last elements.

2.7 Use List and String "Multiplication"

Repeat lists or strings using *.

```
1 print([0] * 3) # Output: [0, 0, 0]
2 print("Hi" * 2) # Output: HiHi
```

2.8 Return Multiple Values

Functions can return multiple values as tuples.

```
1 def get_coords():
2     return 10, 20
3
4 x, y = get_coords()
5 print(x, y) # Output: 10 20
```

2.9 Use Loops and the "else" Keyword

The else clause in loops executes if the loop completes without a break.

```
1 for i in range(5):  
2     if i == 6:  
3         break  
4 else:  
5     print("No break occurred") # Output: No break occurred
```

2.10 Take Advantage of Boolean Values and "not"

Use not to simplify boolean conditions.

```
1 flag = False  
2 if not flag:  
3     print("Flag is False") # Output: Flag is False
```

2.11 Treat Strings as Lists of Characters

Access string characters like a list.

```
1 s = "hello"  
2 print(s[0]) # Output: h
```

2.12 Eliminate Characters by Using "replace"

Remove characters by replacing with an empty string.

```
1 text = "hello , world"  
2 print(text.replace(", ", "")) # Output: hello world
```

2.13 Don't Write Unnecessary Loops

Use list comprehensions or built-in functions instead of loops.

```
1 # Instead of: [sum = 0; for i in lst: sum += i]  
2 total = sum([1, 2, 3]) # Output: 6
```

2.14 Use Chained Comparisons (n < x < m)

Simplify range checks with chained comparisons.

```
1 x = 5  
2 if 0 < x < 10:  
3     print("In range") # Output: In range
```

2.15 Simulate “switch” with a Table of Functions

Use dictionaries to mimic switch statements.

```
1 def add(x, y): return x + y
2 def subtract(x, y): return x - y
3 ops = {"add": add, "sub": subtract}
4 print(ops["add"](5, 3)) # Output: 8
```

2.16 Use the “is” Operator Correctly

Use is for identity (e.g., None) checks, not value comparison.

```
1 x = None
2 if x is None:
3     print("x is None") # Output: x is None
```

2.17 Use One-Line “for” Loops

Use list comprehensions for concise loops.

```
1 squares = [x**2 for x in range(5)]
2 print(squares) # Output: [0, 1, 4, 9, 16]
```

Exercise: Create a list of even numbers using a one-line for loop.

2.18 Squeeze Multiple Statements onto a Line

Use semicolons to combine simple statements.

```
1 x = 1; y = 2; print(x + y) # Output: 3
```

2.19 Write One-Line if/then/else Statements

Use ternary operators for concise conditionals.

```
1 x = 10
2 result = "Even" if x % 2 == 0 else "Odd"
3 print(result) # Output: Even
```

2.20 Create Enum Values with “range”

Use range() for simple enumerations.

```
1 RED, GREEN, BLUE = range(3)
2 print(GREEN) # Output: 1
```

2.21 Reduce the Inefficiency of the “print” Function Within IDLE

Use end="" or join strings to optimize printing.

```
1 print("a", "b", end="") # Output: ab
```

2.22 Place Underscores Inside Large Numbers

Use underscores for readability in large numbers.

```
1 big_num = 1_000_000
2 print(big_num) # Output: 1000000
```

3 Running Python from the Command Line

Run Python scripts using the command line for flexibility.

3.1 Running on a Windows-Based System

Use `python script.py` or `python3 script.py` in Command Prompt.

```
1 # Command: python myscript.py
2 print("Hello from script!") # Output: Hello from script!
```

3.2 Running on a Macintosh System

Use `python3 script.py` in Terminal.

3.3 Using pip or pip3 to Download Packages

Install packages with `pip install package_name`.

```
1 # Command: pip install numpy
```

4 Writing and Using Doc Strings

Document functions with triple-quoted docstrings.

```
1 def add(a, b):
2     """Returns the sum of two numbers."""
3     return a + b
4 print(add.__doc__) # Output: Returns the sum of two numbers.
```

5 Importing Packages

Use `import` to access external modules.

```
1 import math
2 print(math.sqrt(16)) # Output: 4.0
```

6 A Guided Tour of Python Packages

Key packages include `math`, `os`, `sys`, and `numpy`. Explore via `help()` or documentation.

7 Functions as First-Class Objects

Functions can be assigned to variables or passed as arguments.

```
1 def greet(name): return f"Hello , {name}"
2 f = greet
3 print(f("Alice")) # Output: Hello , Alice
```

8 Variable-Length Argument Lists

Handle flexible arguments with `*args` and `**kwargs`.

8.1 The `*args` List

Accept variable positional arguments.

```
1 def sum_nums(*args):
2     return sum(args)
3 print(sum_nums(1, 2, 3)) # Output: 6
```

8.2 The `**kwargs` List

Accept variable keyword arguments.

```
1 def print_info(**kwargs):
2     for k, v in kwargs.items():
3         print(f"{k}: {v}")
4 print_info(name="Alice", age=30) # Output: name: Alice , age: 30
```

9 Decorators and Function Profilers

Decorators wrap functions to add functionality, like timing execution.

```
1 def timer(func):
2     def wrapper(*args, **kwargs):
3         import time
4         start = time.time()
5         result = func(*args, **kwargs)
6         print(f"Time: {time.time() - start} seconds")
7         return result
8     return wrapper
9
10 @timer
11 def slow_function():
12     import time
13     time.sleep(1)
14 slow_function() # Output: Time: ~1.0 seconds
```

10 Generators

Generators yield values lazily, saving memory.

10.1 What's an Iterator?

Iterators provide sequential access to elements using `next()`.

10.2 Introducing Generators

Use `yield` to create generators.

```
1 def fibonacci(n):  
2     a, b = 0, 1  
3     for _ in range(n):  
4         yield a  
5         a, b = b, a + b  
6 print(list(fibonacci(5))) # Output: [0, 1, 1, 2, 3]
```

Exercise: Write a generator for even numbers up to `n`.

11 Accessing Command-Line Arguments

Use `sys.argv` to access command-line inputs.

```
1 import sys  
2 print(sys.argv) # Output: ['script.py', ...]
```

12 Summary

This chapter covered Python shortcuts, command-line usage, package management, and advanced features like decorators and generators. Practice these techniques to write efficient, readable code.

13 Questions for Review

1. Explain the difference between `*args` and `**kwargs`.
2. How does a generator differ from a regular function?
3. What is the purpose of a docstring?

14 Suggested Problems

1. Write a program using a list comprehension to filter odd numbers.
2. Create a decorator that logs function calls to a file.
3. Write a script that accepts command-line arguments and processes them.

Chapter 5: Formatting Text Precisely Tutorial

A Comprehensive Guide to Python Text Formatting Techniques

Tutorial by Ayes Chinmay

July 25, 2025

Contents

1	Introduction	2
2	Formatting with the Percent Sign Operator (%)	2
3	Percent Sign (%) Format Specifiers	2
4	Percent Sign (%) Variable-Length Print Fields	2
5	The Global format Function	3
6	Introduction to the format Method	3
7	Ordering by Position (Name or Number)	3
8	“Repr” Versus String Conversion	3
9	The spec Field of the format Function and Method	3
9.1	Print-Field Width	4
9.2	Text Justification: “fill” and “align” Characters	4
9.3	The “sign” Character	4
9.4	The Leading-Zero Character (0)	4
9.5	Thousands Place Separator	4
9.6	Controlling Precision	4
9.7	“Precision” Used with Strings (Truncation)	5
9.8	“Type” Specifiers	5
9.9	Displaying in Binary Radix	5
9.10	Displaying in Octal and Hex Radix	5
9.11	Displaying Percentages	5
9.12	Binary Radix Example	5
10	Variable-Size Fields	5
11	Summary	6
12	Review Questions	6
13	Suggested Problems	6

1 Introduction

This tutorial explores Python's text formatting techniques, covering the percent sign operator (%), the global format function, and the string format method. These tools allow precise control over how data is displayed, including alignment, precision, and number formatting. This tutorial is based on the structure of Chapter 5, "Formatting Text Precisely," and provides detailed explanations, Python code examples, and practical applications.

2 Formatting with the Percent Sign Operator (%)

The percent sign operator (%), also known as the string formatting operator, is an older method for formatting strings in Python. It uses format specifiers to define how values are inserted into a string.

```
1 # Example: Basic percent sign formatting
2 name = "Alice"
3 age = 25
4 print("Name: %s, Age: %d" % (name, age))
5 # Output: Name: Alice, Age: 25
```

The %s specifier formats strings, while %d formats integers. The values are provided in a tuple after the % operator.

3 Percent Sign (%) Format Specifiers

Format specifiers define the type and style of the data. Common specifiers include:

- %s: String
- %d: Integer
- %f: Floating-point number
- %x: Hexadecimal (lowercase)
- %o: Octal

```
1 # Example: Using different specifiers
2 value = 42
3 print("Integer: %d, Hex: %x, Octal: %o" % (value, value, value))
4 # Output: Integer: 42, Hex: 2a, Octal: 52
```

4 Percent Sign (%) Variable-Length Print Fields

You can specify the width of the print field using a number between % and the specifier. A negative number left-justifies the output.

```
1 # Example: Variable-length fields
2 print("Name: %-10s Age: %5d" % ("Alice", 25))
3 # Output: Name: Alice      Age:      25
```

5 The Global format Function

The format function, introduced in Python 2.6, provides a more flexible way to format strings. It uses placeholders {} and can be customized with format specifications.

```
1 # Example: Using the format function
2 print(format(42, "d"), format(42, "x"), format(3.14159, ".2f"))
3 # Output: 42 2a 3.14
```

6 Introduction to the format Method

The string format method, introduced in Python 3, allows formatting by replacing placeholders {} with values. It is more readable than the % operator.

```
1 # Example: Basic format method
2 name = "Bob"
3 age = 30
4 print("Name: {}, Age: {}".format(name, age))
5 # Output: Name: Bob, Age: 30
```

7 Ordering by Position (Name or Number)

The format method supports positional and named arguments for flexible ordering.

```
1 # Positional ordering
2 print("{1}, {0}".format("first", "second"))
3 # Output: second, first
4
5 # Named arguments
6 print("Name: {name}, Age: {age}".format(name="Bob", age=30))
7 # Output: Name: Bob, Age: 30
```

8 "Repr" Versus String Conversion

The repr specifier (!r) in the format method uses the object's __repr__ method, while the default uses __str__.

```
1 # Example: Repr vs String
2 class Person:
3     def __str__(self): return "Bob"
4     def __repr__(self): return "Person('Bob')"
5
6 p = Person()
7 print("{!r} vs {}".format(p, p))
8 # Output: Person('Bob') vs Bob
```

9 The spec Field of the format Function and Method

The format specification (inside {}) controls aspects like width, alignment, and precision. The syntax is [fill][align][sign][0][width][.][precision][type].

9.1 Print-Field Width

The width specifies the minimum field size.

```
1 # Example: Field width
2 print("{:10}".format("test"))
3 # Output: test
```

9.2 Text Justification: "fill" and "align" Characters

Alignment options are < (left), > (right), (center), *with optional fill characters*.

```
1 # Example: Alignment
2 print("{:*^10}".format("test"))
3 # Output: ***test***
```

Left-aligned

test

Right-aligned

test

Center-aligned

test

9.3 The "sign" Character

The sign (+, -, or space) controls how numbers are displayed.

```
1 # Example: Sign
2 print("{:+} vs {: }".format(42, 42))
3 # Output: +42 vs 42
```

9.4 The Leading-Zero Character (0)

The 0 flag pads numbers with zeros.

```
1 # Example: Leading zeros
2 print("{:05d}".format(42))
3 # Output: 00042
```

9.5 Thousands Place Separator

A comma (,) adds thousands separators.

```
1 # Example: Thousands separator
2 print("{:,}".format(1234567))
3 # Output: 1,234,567
```

9.6 Controlling Precision

The .precision field limits decimal places for floats.

```
1 # Example: Precision
2 print("{:.2f}".format(3.14159))
3 # Output: 3.14
```

9.7 "Precision" Used with Strings (Truncation)

For strings, precision truncates the output.

```
1 # Example: String truncation
2 print("{:.3}".format("hello"))
3 # Output: hel
```

9.8 "Type" Specifiers

Type specifiers like d, f, x, b control the number format.

```
1 # Example: Type specifiers
2 print("{:d} {:x} {:b}".format(42, 42, 42))
3 # Output: 42 2a 101010
```

9.9 Displaying in Binary Radix

The b type displays numbers in binary.

```
1 # Example: Binary
2 print("{:b}".format(42))
3 # Output: 101010
```

9.10 Displaying in Octal and Hex Radix

The o and x/X types display numbers in octal and hexadecimal.

```
1 # Example: Octal and Hex
2 print("{:o} {:x} {:X}".format(42, 42, 42))
3 # Output: 52 2a 2A
```

9.11 Displaying Percentages

The % type converts a number to a percentage.

```
1 # Example: Percentage
2 print("{:%}".format(0.75))
3 # Output: 75.000000%
```

9.12 Binary Radix Example

A practical example of binary formatting:

```
1 # Example: Binary formatting with width
2 print("{:08b}".format(42))
3 # Output: 00101010
```

10 Variable-Size Fields

Variable-size fields use an asterisk (*) to specify width or precision dynamically.

```
1 # Example: Variable width
2 width = 10
3 print("{:*<{width}}".format("test", width=width))
4 # Output: test*****
```

11 Summary

This tutorial covered Python's text formatting techniques, from the percent sign operator to the modern format method. These tools provide precise control over string output, supporting various applications like data reporting and user interfaces.

12 Review Questions

1. What is the difference between %s and %d in the percent sign operator?
2. How does the format method improve upon the % operator?
3. Explain the role of the sign character in formatting numbers.
4. How can you truncate a string using the format method?
5. What is the purpose of the , specifier in number formatting?

13 Suggested Problems

1. Write a program that formats a list of numbers in decimal, hex, and binary with consistent field widths.
2. Create a table of values using the format method with left, right, and center alignment.
3. Format a floating-point number as a percentage with two decimal places.